UNIVERSITY OF WAH (UOW)

DEPARTMENT OF COMPUTER SCIENCE

BSCS 3RD B

REG #: UW-24-CS-BS-088

SUBMITTED BY: SAADI SAFDAR

SUBMITTED TO: MS. WASIFA KANWAL

COURSE TITLE: INTRO TO INFORMATION SECURITY

COURSE CODE: CS – 251 LAB

LAB ASSIGNMENT # 3

Assignment Task 1: Smart Email Security Filter

- The program should ask the user to enter the **subject** and **body** for 3 different emails.
- The script must **search** each email (subject + body) for the following **suspicious words**: urgent, verify, click here, account
- If any of these words appear in the email, they should be **flagged**.

- Count how many suspicious words appear in each email and assign a **risk score** based on the number of red flags:

  ✓ 0 red flags → **Low Risk**
  ✓ 1 red flag → **Medium Risk**
  ✓ 2 or more red flags → **High Risk**

- Display the **result for each email**, showing:

  ✓ The email number
  ✓ The suspicious words found (if any)
  ✓ The total number of red flags
  ✓ The final risk level (Low, Medium, or High)

- After analyzing all 3 emails, print an **overall summary** that includes:

  ✓ Total number of emails analyzed
  ✓ Number of Low Risk, Medium Risk, and High Risk emails

Code:

```
def analyze_emails():
suspicious_words = ["urgent", "verify", "click here", "account"]
risk_counts = {"Low Risk": 0, "Medium Risk": 0, "High Risk": 0}
for i in range(1, 4):
subject = input(f"Enter subject of Email {i}: ").lower()
body = input(f"Enter body of Email {i}: ").lower()
email_text = subject + " " + body
flagged = [word for word in suspicious_words if word in email_text]
red_flags = len(flagged)
if red_flags == 0:
risk = "Low Risk"
elif red_flags == 1:
risk = "Medium Risk"
else:
```

```python
risk = "High Risk"
risk_counts[risk] += 1
print(f"\nEmail {i}:")
print(f"Suspicious words found: {flagged if flagged else 'None'}")
print(f"Total red flags: {red_flags}")
print(f"Risk level: {risk}\n")
print("Overall Summary:")
print("Total emails analyzed: 3")
print(f"Low Risk emails: {risk_counts['Low Risk']}")
print(f"Medium Risk emails: {risk_counts['Medium Risk']}")
print(f"High Risk emails: {risk_counts['High Risk']}")
analyze_emails()
```

lab3.py U

InformationSecurity > lab3.py > analyze_emails

```python

def analyze_emails():
    suspicious_words = ["urgent", "verify", "click here", "account"]
    risk_counts = {"Low Risk": 0, "Medium Risk": 0, "High Risk": 0}
    for i in range(1, 4):
        subject = input(f"Enter subject of Email {i}: ").lower()
        body = input(f"Enter body of Email {i}: ").lower()
        email_text = subject + " " + body
        flagged = [word for word in suspicious_words if word in email_text]
        red_flags = len(flagged)
        if red_flags == 0:
            risk = "Low Risk"
        elif red_flags == 1:
            risk = "Medium Risk"
        else:
            risk = "High Risk"
        risk_counts[risk] += 1
        print(f"\nEmail {i}:")
        print(f"Suspicious words found: {flagged if flagged else 'None'}")
        print(f"Total red flags: {red_flags}")
        print(f"Risk level: {risk}\n")
    print("Overall Summary:")
    print("Total emails analyzed: 3")
    print(f"Low Risk emails: {risk_counts['Low Risk']}")
    print(f"Medium Risk emails: {risk_counts['Medium Risk']}")
    print(f"High Risk emails: {risk_counts['High Risk']}")
analyze_emails()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saadi@Saadi-PC:~/Python Programs$ /usr/bin/python3 "/home/saadi/Python Programs/InformationSecurity/lab3.py"
Enter subject of Email 1: Urgent account verification needed
Enter body of Email 1: Please click here to verify your account immediately.

Email 1:
Suspicious words found: ['urgent', 'verify', 'click here', 'account']
Total red flags: 4
Risk level: High Risk

Enter subject of Email 2: Meeting schedule confirmation
Enter body of Email 2: Let me know if you can attend the meeting tomorrow.

Email 2:
Suspicious words found: None
Total red flags: 0
Risk level: Low Risk

Enter subject of Email 3: Verify your details
Enter body of Email 3: This is a friendly reminder to verify your account.

Email 3:
Suspicious words found: ['verify', 'account']
Total red flags: 2
Risk level: High Risk

Overall Summary:
Total emails analyzed: 3
Low Risk emails: 1
Medium Risk emails: 0
High Risk emails: 2
saadi@Saadi-PC:~/Python Programs$
```

Assignment Task 2:

Write a Python program that has following functions:

- **sender(msg)** → compute checksum = sum(ord(c) for c in msg) as integer; return a string packet: "<msg>|<checksum>" and print it.
- **network(packet, tamper=False)** → if tamper is True, change the message text (e.g., replace "abc123" with "ab123") but **do not update checksum**; print what attacker sends on.
- **receiver(packet)** → split on "|" to get msg and checksum_str; compute checksum of received msg and compare to checksum_str. If equal print "OK", else print "Tampered!".

Demonstrate both cases: one with tamper=False and another with tamper=True.

Code:

```
def sender(msg):
checksum = sum(ord(c) for c in msg)
packet = f"{msg}|{checksum}"
print("Sender sends:", packet)
return packet

def network(packet, tamper=False):
if tamper:
tampered_msg = packet.split('|')[0].replace("abc123", "ab123")
checksum = packet.split('|')[1]
tampered_packet = f"{tampered_msg}|{checksum}"
print("Attacker sends on:", tampered_packet)
return tampered_packet
else:
print("Network sends:", packet)
return packet

def receiver(packet):
msg, checksum_str = packet.split('|')
checksum = sum(ord(c) for c in msg)
print("Receiver gets packet:", packet)
if str(checksum) == checksum_str:
print("OK")
else:
print("Tampered!")

# Demonstration without tampering
pkt = sender("abc123")
pkt = network(pkt, tamper=False)
receiver(pkt)

print() # Separator
```

```python
# Demonstration with tampering
pkt = sender("abc123")
pkt = network(pkt, tamper=True)
receiver(pkt)
```

lab3.py u ×

InformationSecurity > lab3.py > ...

```python
def sender(msg):
    checksum = sum(ord(c) for c in msg)
    packet = f"{msg}|{checksum}"
    print("Sender sends:", packet)
    return packet

def network(packet, tamper=False):
    if tamper:
        tampered_msg = packet.split('|')[0].replace("abc123", "ab123")
        checksum = packet.split('|')[1]
        tampered_packet = f"{tampered_msg}|{checksum}"
        print("Attacker sends on:", tampered_packet)
        return tampered_packet
    else:
        print("Network sends:", packet)
        return packet

def receiver(packet):
    msg, checksum_str = packet.split('|')
    checksum = sum(ord(c) for c in msg)
    print("Receiver gets packet:", packet)
    if str(checksum) == checksum_str:
        print("OK")
    else:
        print("Tampered!")

# Demonstration without tampering
pkt = sender("abc123")
pkt = network(pkt, tamper=False)
receiver(pkt)

print()  # Separator

# Demonstration with tampering
pkt = sender("abc123")
pkt = network(pkt, tamper=True)
receiver(pkt)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saadi@Saadi-PC:~/Python Programs$ /usr/bin/python3 "/home/saadi/Python Programs/InformationSecurity/lab3.py"
Sender sends: abc123|444
Network sends: abc123|444
Receiver gets packet: abc123|444
OK

Sender sends: abc123|444
Attacker sends on: ab123|444
Receiver gets packet: ab123|444
Tampered!
saadi@Saadi-PC:~/Python Programs$
```

Assignment Task 3:

Write a Python program that:

- Given a list requests where each request is a tuple (req_id, source) — source can be an IP-like string (e.g., "10.0.0.5").
- Service capacity (max processed requests overall) = capacity.
- Per-source maximum allowed requests = per_source_limit. If a source sends more than per_source_limit requests, mark that source **blocked** and skip its future requests.

- Process requests in order. For each request:

  ✓ If source is blocked → print Blocked <req_id> from <source> and skip.
  ✓ Else if processed < capacity → print Processing <req_id> from <source> and count it.
  ✓ Else → print Server overloaded! and stop processing.

- At the end print a short summary: processed count and blocked sources.

Code:

```
def process_requests(requests, capacity, per_source_limit):
processed_count = 0
blocked_sources = set()
source_count = {}
for req_id, source in requests:
if source in blocked_sources:
print(f"Blocked {req_id} from {source}")
continue
if processed_count >= capacity:
print("Server overloaded!")
break
count = source_count.get(source, 0)
if count >= per_source_limit:
blocked_sources.add(source)
print(f"Blocked {req_id} from {source}")
continue
print(f"Processing {req_id} from {source}")
processed_count += 1
source_count[source] = count + 1
print("\nSummary:")
print(f"Total processed requests: {processed_count}")
print(f"Blocked sources: {', '.join(blocked_sources) if blocked_sources else 'None'}")


requests = [
(1, "10.0.0.1"),
(2, "10.0.0.2"),
```

```
(3, "10.0.0.1"),
(4, "10.0.0.1"),
(5, "10.0.0.3"),
(6, "10.0.0.2"),
]

capacity = 5
per_source_limit = 2

process_requests(requests, capacity, per_source_limit)
```

lab3.py

InformationSecurity > lab3.py > ...

```python
def process_requests(requests, capacity, per_source_limit):
    processed_count = 0
    blocked_sources = set()
    source_count = {}
    for req_id, source in requests:
        if source in blocked_sources:
            print(f"Blocked {req_id} from {source}")
            continue
        if processed_count >= capacity:
            print("Server overloaded!")
            break
        count = source_count.get(source, 0)
        if count >= per_source_limit:
            blocked_sources.add(source)
            print(f"Blocked {req_id} from {source}")
            continue
        print(f"Processing {req_id} from {source}")
        processed_count += 1
        source_count[source] = count + 1
    print("\nSummary:")
    print(f"Total processed requests: {processed_count}")
    print(f"Blocked sources: {', '.join(blocked_sources) if blocked_sources else 'None'}")
requests = [
    (1, "10.0.0.1"),
    (2, "10.0.0.2"),
    (3, "10.0.0.1"),
    (4, "10.0.0.1"),
    (5, "10.0.0.3"),
    (6, "10.0.0.2"),
]
capacity = 5
per_source_limit = 2
process_requests(requests, capacity, per_source_limit)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saadi@Saadi-PC:~/Python Programs$  /usr/bin/env /usr/bin/python3 /home/saadi/.vscode/extensions/ms-python.debugpy-
\ Programs/InformationSecurity/lab3.py
Processing 1 from 10.0.0.1
Processing 2 from 10.0.0.2
Processing 3 from 10.0.0.1
Blocked 4 from 10.0.0.1
Processing 5 from 10.0.0.3
Processing 6 from 10.0.0.2

Summary:
Total processed requests: 5
Blocked sources: 10.0.0.1
saadi@Saadi-PC:~/Python Programs$
```

Simulating Common Attacks:

1. Phishing (simulation (local email analysis))

```
email = {
"from": "support@email.com",
"to": "stu@edu.pk",
"subject": "URGENT: Meeting Reminder",
"body": "Don't forget about the meeting tomorrow at 10 AM.",
}
print("Emails Recieved:")
for key, value in email.items():
print(f"{key}: {value}")
warnings = []
if "urgent" in email["subject"].lower():
warnings.append("The email subject contains the word 'URGENT'.")
if "http://" in email["body"] or "https://" in email["body"]:
warnings.append("The email contains a non HTTPS link.")
if email["from"].split("@")[-1].count("-") > 0 or len(email["from"].split("@")[-1].split(".")) < 2:
warnings.append("The sender's email domain looks suspicious.")
if warnings:
print("\nWarnings:")
for w in warnings:
print("- ", w)
else:
print("\n - No obivous threats detected in the email. - ", "Still be cautious!")
```

lab4.py U ×

InformationSecurity > lab4.py > email

```python
email = {
    "from": "support@email.com",

    "to": "stu@edu.pk",
    "subject": "URGENT: Meeting Reminder",
    "body": "Don't forget about the meeting tomorrow at 10 AM.",
}
print("Emails Recieved:")
for key, value in email.items():
    print(f"{key}: {value}")
warnings = []
if "urgent" in email["subject"].lower():
    warnings.append("The email subject contains the word 'URGENT'.")
if "http://" in email["body"] or "https://" in email["body"]:
    warnings.append("The email contains a non HTTPS link.")
if email["from"].split("@")[-1].count("-") > 0 or len(email["from"].split("@")[-1].split(".")) < 2
    warnings.append("The sender's email domain looks suspicious.")
if warnings:
    print("\nWarnings:")
    for w in warnings:
        print("- ", w)
else:
    print("\n - No obivous threats detected in the email. - ", "Still be cautious!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
saadi@Saadi-PC:~/Python Programs$ /usr/bin/python3 "/home/saadi/Python Programs/InformationSecurity/lab4.py"
Emails Recieved:
from: support@email.com
to: stu@edu.pk
subject: URGENT: Meeting Reminder
body: Don't forget about the meeting tomorrow at 10 AM.

Warnings:
-  The email subject contains the word 'URGENT'.
Server starting capacity: 5
```

Ln 1, Col 1 Spaces: 4 UTF-8 LF {} Python 3.12.3 Go Live

2. Denial of Service (DoS)

```python
import time
from queue import Queue
SERVICE_CAPACITY = 5
requests = Queue()
for i in range(8):
requests.put(f"Requests_{i}")
processed = 0
print("Server starting capacity:" , SERVICE_CAPACITY)
while not requests.empty():
if processed >= SERVICE_CAPACITY:
print("Server Overloaded: response time increasing...")
time.sleep(1)
req = requests.get()
print("Processing", req)
processed += 1
```

time.sleep(0.1)

lab4.py U ×

InformationSecurity > lab4.py > email

```python

import time
from queue import Queue

SERVICE_CAPACITY = 5
requests = Queue()

for i in range(8):
    requests.put(f"Requests_{i}")

processed = 0
print("Server starting capacity:" , SERVICE_CAPACITY)
while not requests.empty():
    if processed >= SERVICE_CAPACITY:
        print("Server Overloaded: response time increasing...")
        time.sleep(1)
    req = requests.get()
    print("Processing", req)
    processed += 1
    time.sleep(0.1)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saadi@Saadi-PC:~/Python Programs$ /usr/bin/python3 "/home/saadi/Python Pr
Server starting capacity: 5
Processing Requests_0
Processing Requests_1
Processing Requests_2
Processing Requests_3
Processing Requests_4
Server Overloaded: response time increasing...
Processing Requests_5
Server Overloaded: response time increasing...
Processing Requests_6
Server Overloaded: response time increasing...
Processing Requests_7
```

Ln 1, Col 1 Spaces: 4

3. Password Attack

```
users = {"student" : "1234pass"}
def authenticate(username, password):
return users.get(username) == password
```

```
wordlist = ["1234", "pass1234", "password", "1234pass"]
for guess in wordlist:
print("Trying password:", guess)
if authenticate("student", guess):
print("Password found: ", guess)
break
else:
print("Password not found in the tested wordlist")
```

lab4.py U ×

InformationSecurity > lab4.py > email

```
46
47  users = {"student" : "1234pass"}
48
49  def authenticate(username, password):
50      return users.get(username) == password
51
52  wordlist = ["1234", "pass1234", "password", "1234pass"]
53
54  for guess in wordlist:
55      print("Trying password:", guess)
56      if authenticate("student", guess):
57          print("Password found: ", guess)
58          break
59  else:
60      print("Password not found in the tested wordlist")
61
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saadi@Saadi-PC:~/Python Programs$ /usr/bin/python3 "/home/saadi/
mationSecurity/lab4.py"
Trying password: 1234
Trying password: pass1234
Trying password: password
Trying password: 1234pass
Password found:  1234pass
```

4. Man-in-the-Middle (MITM)

```
def sender(msg):
print("[Sender] sending:", msg)
return msg
def network_path(message):
print("[Hacker] intercepted:", message)
return message
def receiver(msg):
print("[Receiver] recieiving:", msg)
if __name__ == "__main__":
s = sender("Sensitive: password123")
```

```
r = network_path(s)
receiver(r)
```

lab4.py U ×

InformationSecurity > lab4.py > email

```python
def sender(msg):
    print("[Sender] sending:", msg)
    return msg
def network_path(message):
    print("[Hacker] intercepted:", message)
    return message
def receiver(msg):
    print("[Receiver] recieiving:", msg)

if __name__ == "__main__":
    s = sender("Sensitive: password123")
    r = network_path(s)
    receiver(r)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saadi@Saadi-PC:~/Python Programs$ /usr/bin/python3
mationSecurity/lab4.py"
[Sender] sending: Sensitive: password123
[Hacker] intercepted: Sensitive: password123
[Receiver] recieiving: Sensitive: password123
```